UNIVERSIDAD DON BOSCO



Escuela de Computación

SISTEMA DE GESTIÓN Y RESERVA DE HABITACIONES HOTELERAS

Fase Final del Proyecto

Integrantes del equipo y roles:

Estudiante	**Código**	**Rol**
Manuel de Jesús Hernández Escamilla	HE252995	Coordinador/Desarrollador
Manuel Enrique Miranda Rivas	MR252985	Desarrollador FrontEnd
Walter Fabricio Salazar Consuegra	SC252970	Desarrollador BackEnd

Catedrático:
Ing. Mario Alvarado

Asignatura:
Desarrollo de Aplicaciones con Web Frameworks

Fecha de entrega:
29 de octubre de 2026

Índice

3. Descripción del Proyecto **3**
3.1 Objetivo General **3**
3.2 Objetivos Específicos **3**
4. Gestión Integral del Proyecto **4**
4.1 Gestión de la Integración **4**
4.2 Gestión del Alcance **4**
4.3 Gestión del Cronograma **6**
a) Calendario del Proyecto 6
b) Hitos y Plazos Clave 7
4.4 Gestión de Costos **8**
4.5 Gestión de la Calidad **9**
a) Revisión de Código 9
b) Estándares de Codificación 9
4.6 Gestión de los Recursos Humanos **9**
a) Asignación de Roles 9
b) Colaboración y Comunicación 9
4.7 Gestión de la Comunicación **10**
a) Informes Semanales de Progreso 10
b) Canales de Resolución de Dudas 10
4.8 Gestión de Adquisición y Cierre **10**
5. Descripción Técnica **11**
5.1 Prototipo de FrontEnd **11**
a) Pantalla 1 11
b) Pantalla 2 12
c) Pantalla 3 13
d) Pantalla 4 14
e) Pantalla 5 15
f) Pantalla 6 16
g) Pantalla 7 17
h) Pantalla 8 18
i) Pantalla 9 19
j) Pantalla 10 20
k) Pantalla 11 21
5.2 Gestión de la Base de Datos **22**
a) Estructura de la Base de Datos 22
b) Diccionario de datos 24
1. Tabla ROL 24
2. Tabla USUARIO 24
3. Tabla HOTEL 25
4. Tabla SUCURSAL 25
5. Tabla TIPO_HABITACION 25
7. Tabla TARIFA 26
8. Tabla DISPONIBILIDAD 27
9. Tabla HUESPED 27
10. Tabla SERVICIO 28
11. Tabla RESERVA 28
12. Tabla RESERVA_HUESPED 29
13. Tabla RESERVA_SERVICIO 29
14. Tabla PAGO 30
15. Tabla POLITICA_CANCELACION 31
16. Tabla CANCELACION 31
17. Tabla RECLAMO 32
6. Resultados y Conclusiones **33**
6.1 Resultados Obtenidos **33**

6.2 Conclusiones 33
7. Anexos y Documentación de Apoyo..... 34

3. Descripción del Proyecto

3.1 Objetivo General

Desarrollar un sistema de administración y reserva de habitaciones hoteleras. El backend se construirá en Java con Spring Boot y Spring Data JPA, sobre una base de datos Microsoft SQL Server, y expondrá una API REST con enfoque SOA. El frontend en HTML, CSS y JavaScript consumirá esa API para cubrir la búsqueda, reserva, pago y gestión de habitaciones, y reducir los errores manuales en la operación del hotel.

3.2 Objetivos Específicos

- Diseñar e implementar un modelo de base de datos relacional en Microsoft SQL Server, utilizando Lucidchart para el diagrama entidad-relación, que garantice la integridad y consistencia de la información relacionada con hoteles, habitaciones, huéspedes, reservas y pagos.

- Diseñar la arquitectura de la API REST en Java con Spring Boot, apoyada en Spring Data JPA/Hibernate para la persistencia de datos y Lombok para la reducción de código repetitivo, definiendo los endpoints necesarios para cubrir el proceso completo de reserva: búsqueda de disponibilidad, selección de habitación, registro de datos del huésped y confirmación de pago.
- Diseñar un prototipo de interfaz de usuario en HTML, CSS y JavaScript, elaborado previamente en Figma con un mínimo de 10 maquetas, que sirva como base funcional y visual para la futura implementación e integración con la API REST.
- Implementar el consumo de la API REST desde el frontend mediante peticiones HTTP con fetch en JavaScript, permitiendo la comunicación directa entre las pantallas de la aplicación y los endpoints del backend.
- Diseñar un sistema de gestión de usuarios y roles, mediante una entidad Usuario con un campo de rol asociado, controlando a nivel de lógica de negocio el acceso a las distintas funcionalidades del sistema según el perfil del usuario.
- Planificar la implementación de la gestión de cancelaciones y modificaciones de reservas, aplicando las políticas de cancelación definidas por el hotel y el cálculo de cargos o reembolsos correspondientes.
- Diseñar el módulo de gestión de reclamos y solicitudes de los huéspedes, permitiendo su registro, seguimiento y resolución dentro del sistema.
- Establecer la estrategia de integración entre el frontend y el backend, utilizando Swagger UI para la validación de endpoints antes de conectarlos, de manera que en la Fase 2 el sistema pueda implementarse y conectarse de forma completa y funcional.
- Gestionar el control de versiones del proyecto mediante GitHub y el seguimiento de tareas mediante Trello, asegurando una colaboración ordenada entre los integrantes del equipo.

4. Gestión Integral del Proyecto

4.1 Gestión de la Integración

El equipo gestiona el código y las tareas con GitHub y Trello, mediante un flujo colaborativo directo en la rama main y seguimiento por tarjetas.

Para el control de versiones se utilizará GitHub, donde se creó un repositorio para el proyecto (sistema-hotel) con tres carpetas principales: backend (API REST en Spring Boot), frontend (interfaz en HTML, CSS y JavaScript) y docs (documento técnico, diagramas y prototipos). Los tres integrantes del equipo tienen acceso como colaboradores. Todo el equipo trabaja directamente sobre la rama main, sin ramas adicionales ni Pull Requests. Cada integrante sube únicamente los archivos de su módulo, actualiza su copia con git pull antes de cada commit y revisa git status antes de subir sus cambios, para evitar conflictos entre los avances del equipo.

Para el seguimiento del proyecto se utilizará Trello, en un tablero llamado *"Proyecto Fase 1 y Fase 2 - Sistema de Hotel"*, organizado en tres columnas: *Lista de tareas*, *En proceso* y *Hecho*. Cada punto del documento y cada módulo del sistema se representa como una tarjeta individual, asignada a un responsable, que se traslada de columna conforme avanza su desarrollo.

La metodología ágil que se emplea es Scrum, adaptada a un equipo de tres personas y a los tiempos académicos del curso. El trabajo se organiza en sprints cortos, con revisión periódica de avances en el tablero de Trello. Los roles definidos dentro del equipo son:

- **Coordinador(a) / Scrum Master:** organiza el repositorio y el tablero, da seguimiento a las tareas y consolida la documentación final.
- **Desarrollador(a) FrontEnd:** diseña los prototipos de interfaz en Figma y, en la Fase 2, implementa el consumo de la API REST en HTML, CSS y JavaScript.
- **Desarrollador(a) BackEnd:** diseña la base de datos en Microsoft SQL Server y el diagrama entidad-relación en Lucidchart, y en la Fase 2, implementa la lógica de negocio y los endpoints de la API REST con Spring Boot.

4.2 Gestión del Alcance

El presente proyecto tiene como alcance el desarrollo de un sistema de administración y reserva de habitaciones para un hotel o cadena hotelera, cubriendo tanto la lógica de negocio (backend) como una propuesta de interfaz (frontend).

Dentro del alcance (in-scope) se incluye:

- Registro y gestión de hoteles y sucursales.
- Administración de habitaciones, tipos de habitación, tarifas y servicios asociados.
- Gestión de disponibilidad y períodos de alojamiento.
- Registro y administración de huéspedes.
- Proceso completo de reserva: búsqueda de disponibilidad, selección de habitación, ingreso de datos del huésped, confirmación y pago.
- Gestión de cancelaciones y modificaciones de reservas, incluyendo cálculo de cargos o reembolsos según políticas del hotel.
- Procesamiento de reclamos y solicitudes de los huéspedes.
- Generación de estadísticas básicas (ocupación, reservaciones, cancelaciones e ingresos).
- Gestión de usuarios y roles (administrador, recepcionista y huésped), mediante una entidad Usuario con un campo de rol, controlando el acceso a las funcionalidades según el perfil.

Fuera del alcance (out-of-scope) en esta fase se encuentra:

- La implementación de pasarelas de pago reales (se simulará el registro del pago, sin procesar transacciones bancarias).
- Aplicaciones móviles nativas (el proyecto se limita a una API REST consumida por un frontend web).
- Integraciones con sistemas externos de terceros (motores de reservas externos, redes de distribución hotelera, etc.).
- Funcionalidades de facturación electrónica o cumplimiento fiscal.

Reglas para cambios de alcance: Cualquier modificación a las funcionalidades aquí descritas ya sea para agregar o eliminar una funcionalidad deberá ser analizada y acordada por todo el grupo, documentada como tarjeta nueva en el tablero de Trello y, de ser necesario, decirle al docente antes de su implementación, para evitar afectar la evaluación del proyecto. Además, según las indicaciones del proyecto, una vez hecho el equipo de trabajo este no podrá disolverse ni modificarse.

4.3 Gestión del Cronograma

a) Calendario del Proyecto

Fase 1: Diseño y documentación técnica (del 20 al 30 de agosto de 2026)

Actividades	**Fechas**
Configuración de roles, GitHub y Trello	20–21 agosto
Definición de objetivos, alcance y stack tecnológico	21–22 agosto
Redacción de gestión del proyecto (4.1–4.8)	23–28 agosto
Diseño de prototipos (Figma) y modelo de base de datos	23–29 agosto
Consolidación, formato y revisión final del documento	29 agosto
Entrega del documento en Aula Digital	30 agosto

Fase 2: Desarrollo completo y entrega del sistema (del 31 de agosto al 29 de octubre de 2026)

Sprint	**Actividades**	**Fechas**
Sprint 1	Configuración del entorno (Spring Boot, SQL Server, repositorio) y desarrollo de endpoints base (CRUD de hoteles, habitaciones, huéspedes)	31 ago – 20 sep
Sprint 2	Implementación del proceso completo de reserva (búsqueda, selección, confirmación, pago)	21 sep – 27 sep
Sprint 3	Implementación de cancelaciones, servicios adicionales, reclamos y estadísticas	28 sep – 04 oct
Sprint 4	Implementación del frontend (HTML/CSS/JS) y consumo de la API REST	05 oct – 17 oct
Sprint 5	Pruebas de integración (Swagger UI y frontend), corrección de errores y ajustes al documento técnico	18 oct – 26 oct
Sprint 6	Consolidación final y entrega del proyecto de cátedra	27 oct – 29 oct

b) Hitos y Plazos Clave

Hito	**Fecha**
Conformación del equipo	03 de julio 2026
Asignación del proyecto de cátedra	20 de agosto 2026
Definición de objetivos, alcance y herramientas tecnológicas	24 de agosto 2026
Envío del documento técnico (Fase 1)	30 de agosto 2026
Configuración del entorno de desarrollo	20 de septiembre 2026
Implementación completa de la API REST	04 de octubre de 2026
Integración FrontEnd - BackEnd	15 de octubre de 2026
Finalización y entrega del proyecto (Fase 2)	19–29 de octubre de 2026

4.4 Gestión de Costos

El proyecto usa herramientas y servicios con planes gratuitos, por lo que el costo directo es $0. Si el sistema pasara a producción, habría que contratar planes de pago. A continuación se detalla el presupuesto estimado.

Recurso	Plan usado (Fase 1-2)	Costo actual	Costo si se escala	Observaciones
Hosting de base de datos (SQL Server local, desarrollo)	Gratuito	$0.00	Desde $4.90/mes (nivel DTU básico) hasta más, según vCores/almacenamiento	Base de datos local en SQL Server (localhost:1433) para desarrollo. Si se despliega en la nube, el plan gratuito de Azure SQL Database incluye 100,000 segundos-vCore y 32 GB de datos gratis por mes
Lucidchart	Gratuito	$0.00	Plan Individual: $7.95/mes (anual)	El plan gratuito permite hasta 3 documentos editables, suficiente para el diagrama entidad-relación del proyecto
Figma	Gratuito	$0.00	Plan Professional: $16.00/mes por editor (anual) o $20/mes (mensual)	El plan gratuito limita a 3 archivos de diseño; suficiente para los 10 prototipos si se organizan en un solo archivo
GitHub	Gratuito	$0.00	Plan Team: $4.00/mes por usuario	El plan gratuito cubre repositorios privados/públicos ilimitados para equipos pequeños
Trello	Gratuito	$0.00	Plan Estándar: $5.00/mes por usuario	Plan gratuito, suficiente para un tablero con pocos miembros
Dominio propio	No aplica en esta fase	$0.00	Aprox. $20–35/año (.com)	Solo si el sistema se convierte en una producción real
Total estimado (Fase 1-2)		$0.00		Costo cero, debido al uso de los planes gratuitos
Total estimado si se escala a producción		≈ $37.85/mes (sin dominio)		Sumando Azure SQL Database, Lucidchart, Figma, GitHub Team y Trello Standard (sin dominio).

4.5 Gestión de la Calidad

a) Revisión de Código

Todo cambio de código se sube directamente a la rama main del repositorio, sin Pull Requests. Antes de cada commit, cada integrante actualiza su copia con git pull, revisa con git status que solo aparezcan los archivos de su módulo y luego sube sus cambios con git push. Cada integrante revisa su propio código antes de subirlo, verificando que cumpla su función, que no rompa funcionalidades existentes y que siga la separación en capas: controlador, servicio, repositorio, modelo.

b) Estándares de Codificación

El equipo seguirá las convenciones estándar de Java y Spring Boot:

- Nomenclatura en inglés para clases, métodos y variables (camelCase para métodos/variables, PascalCase para clases).
- Organización del backend en paquetes por capa: controller, service, repository, model/entity, dto.
- Uso de anotaciones estándar de Spring Boot (@RestController, @Service, @Repository, @Entity) para mantener una estructura clara y desacoplada.
- Comentarios explicativos donde la lógica no sea evidente solo de vista, evitando el código redundante o sin uso.

4.6 Gestión de los Recursos Humanos

a) Asignación de Roles

Integrante	**Rol**
Manuel de Jesús Hernández Escamilla	Coordinador(a) / Documentación de Endpoints y consolidación del documento / Backend (Java Spring Boot)
Manuel Enrique Miranda Rivas	FrontEnd (prototipos en Figma, HTML, CSS y JavaScript) y Backend (Java Spring Boot)
Walter Fabricio Salazar Consuegra	Diseño de Base de Datos y Backend (Java Spring Boot)

b) Colaboración y Comunicación

El equipo se comunicará principalmente a través de un grupo de WhatsApp, utilizado para coordinación diaria, dudas rápidas y compartir avances. De forma complementaria, el equipo en ocasiones se reúne en persona para discutir algunas situaciones en donde se necesite tomar una decisión de forma conjunta. El Aula Digital se usará como canal oficial para consultas directas con el docente, y para compartir recursos o documentos también se utilizará Google Drive según la conveniencia del equipo. El seguimiento de tareas se realiza mediante el tablero de Trello, donde cada integrante actualiza el estado de sus tarjetas asignadas.

4.7 Gestión de la Comunicación

a) Informes Semanales de Progreso

El coordinador del equipo actualizará el estado general del proyecto en una breve reunión semanal por Google Meet, revisando el tablero de Trello y verificando que las tarjetas estén de acuerdo con el avance real de cada integrante.

b) Canales de Resolución de Dudas

Las dudas internas del equipo se resuelven por WhatsApp o de forma presencial. Las consultas dirigidas al docente se realizan a través del Aula Digital o al final de alguna clase de teoría y, sino otra forma sería a través del correo institucional cuando se requiera una respuesta un poco más formal.

4.8 Gestión de Adquisición y Cierre

Como parte del cierre de la Fase 1, se generará la documentación final del proyecto, compuesta por el presente documento en formato PDF y un archivo README.md en el repositorio de GitHub, que incluye:

- Nombre del proyecto y descripción general.
- Integrantes del equipo y sus roles.
- Tecnologías utilizadas.
- Enlace al tablero de seguimiento en Trello.

Documentación Final:

- Repositorio de GitHub: https://github.com/Manu-Hdez29/sistema-hotel
- Tablero de Trello: https://trello.com/b/7yjW5JqD/proyecto-fase-1-sistema-de-hotel

Verificación de Cierre

Antes de la entrega final, el coordinador verificará que:

- Todos los puntos del documento (1 al 7) estén completos.
- El formato cumpla lo solicitado (márgenes 2 cm, interlineado 1.15, Calibri 10, texto justificado).
- El repositorio de GitHub contenga el PDF final, el README actualizado y las carpetas backend, frontend y docs.
- El tablero de Trello refleje el estado real de todas las tareas (movidas a "Hecho").

5. Descripción Técnica

5.1 Prototipo de FrontEnd

Figma fue la herramienta principal para el diseño visual del prototipo de alta fidelidad. Los colores son sobrios para que el sistema transmita confianza: el color primario es #1E3A5F, el de acento #2F80ED, el fondo #F7F8FA y las superficies son blancas con bordes #E1E4E8.

El flujo tiene dos recorridos. El cliente empieza en la búsqueda de alojamiento y avanza por un stepper de cinco pasos: selección de habitación, datos personales, resumen, pago (simulado) y confirmación. También tiene una sección con su historial de reservas. El administrador tiene un panel para gestionar hoteles, sucursales y tipos de habitación.

a) Pantalla 1

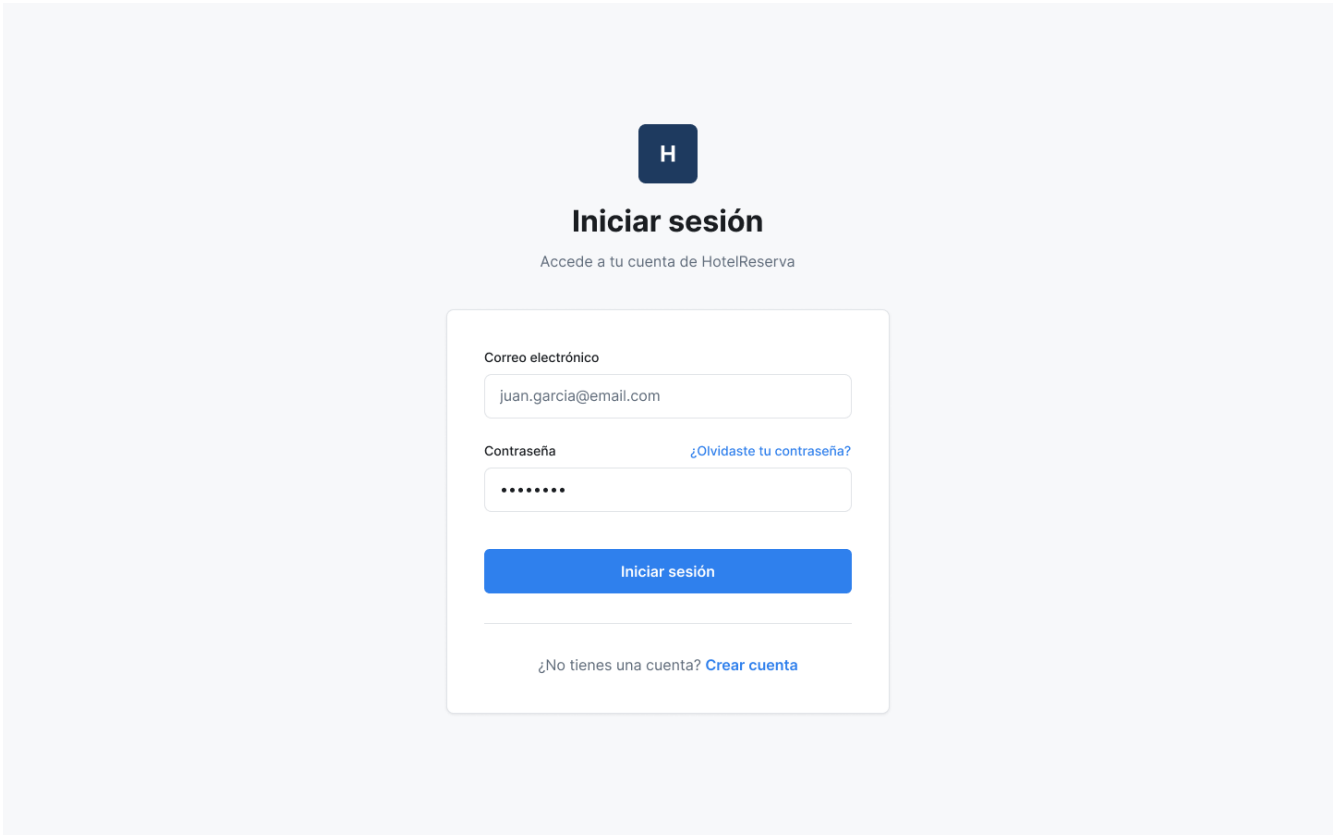


Figura 1. AUTH - Login

b) Pantalla 2

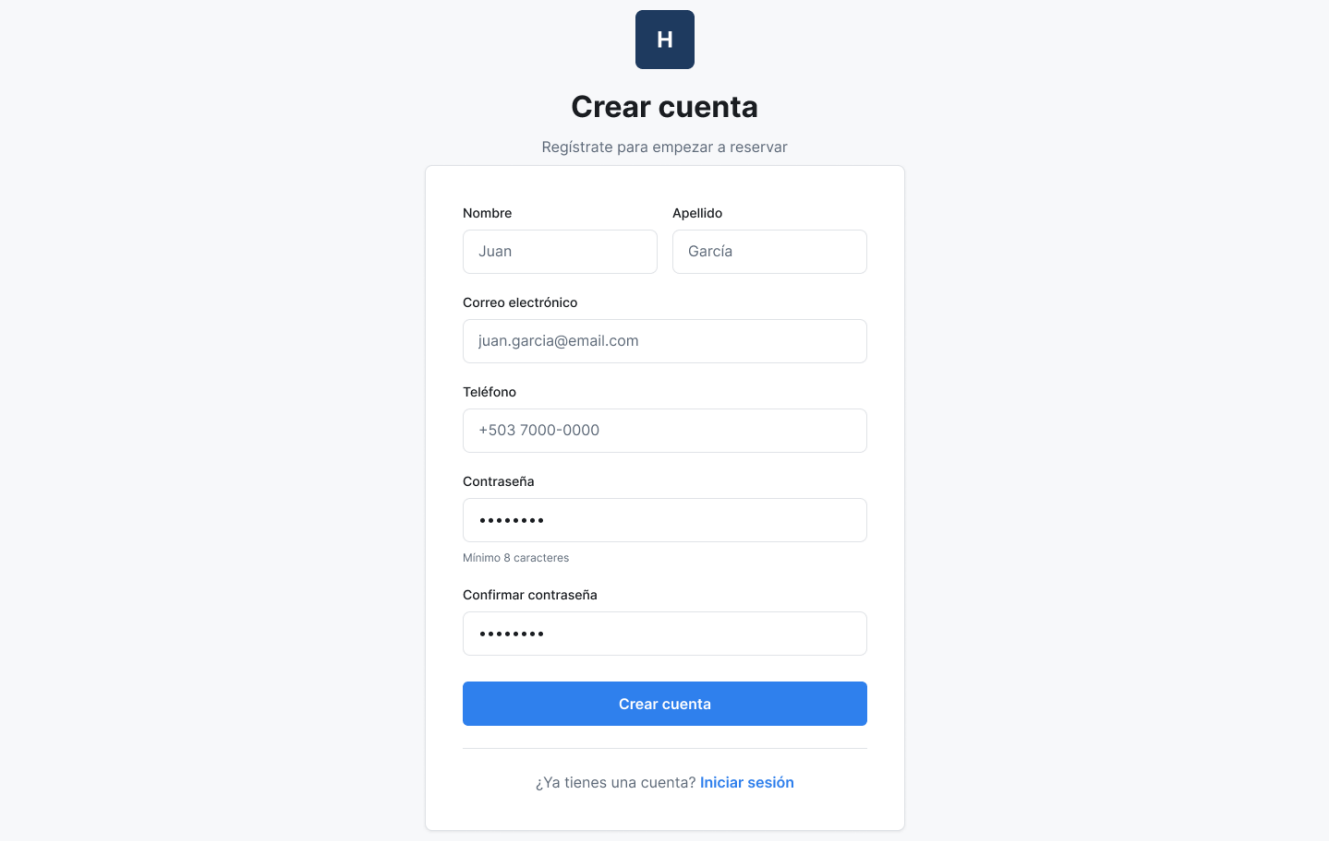


Figura 2. AUTH - Registro

c) Pantalla 3

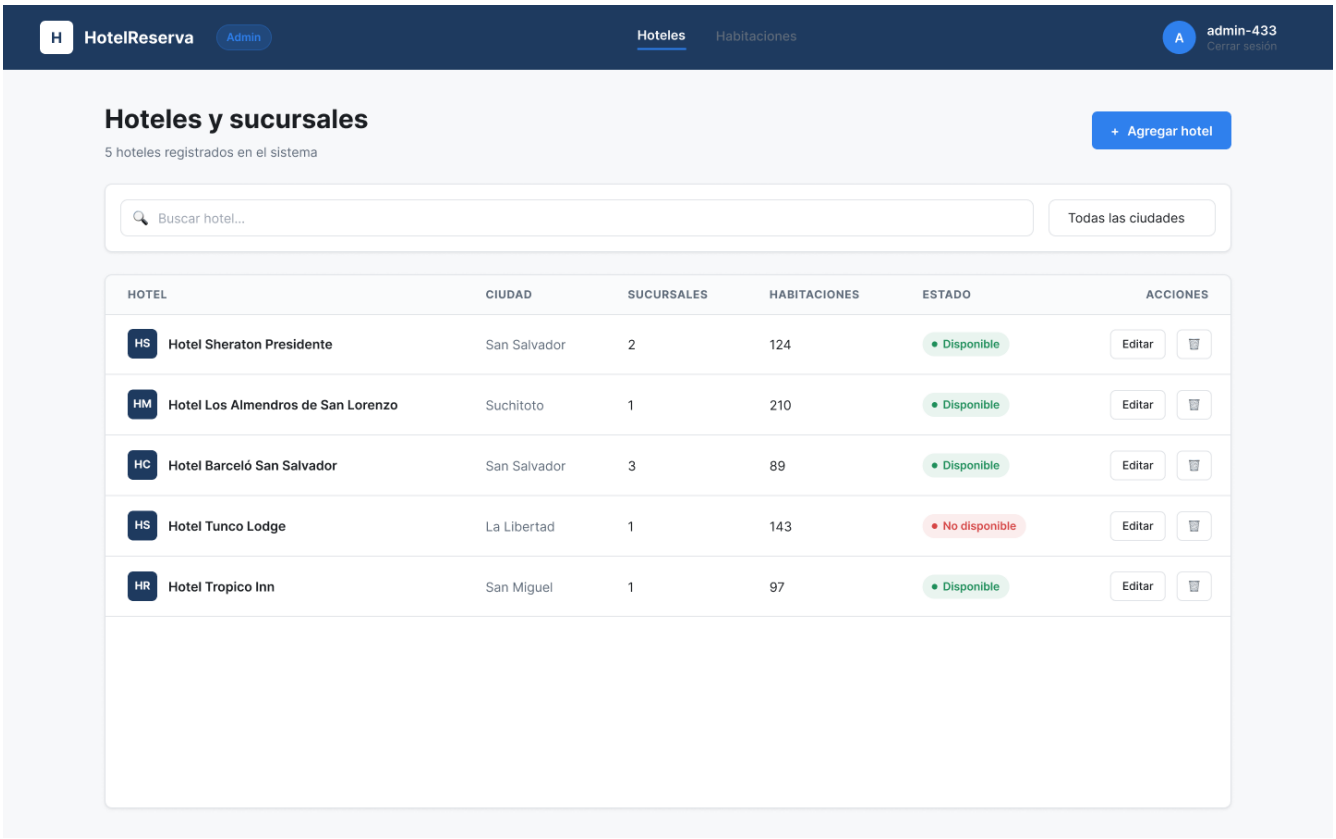


Figura 3. ADMIN - Hoteles y sucursales

d) Pantalla 4

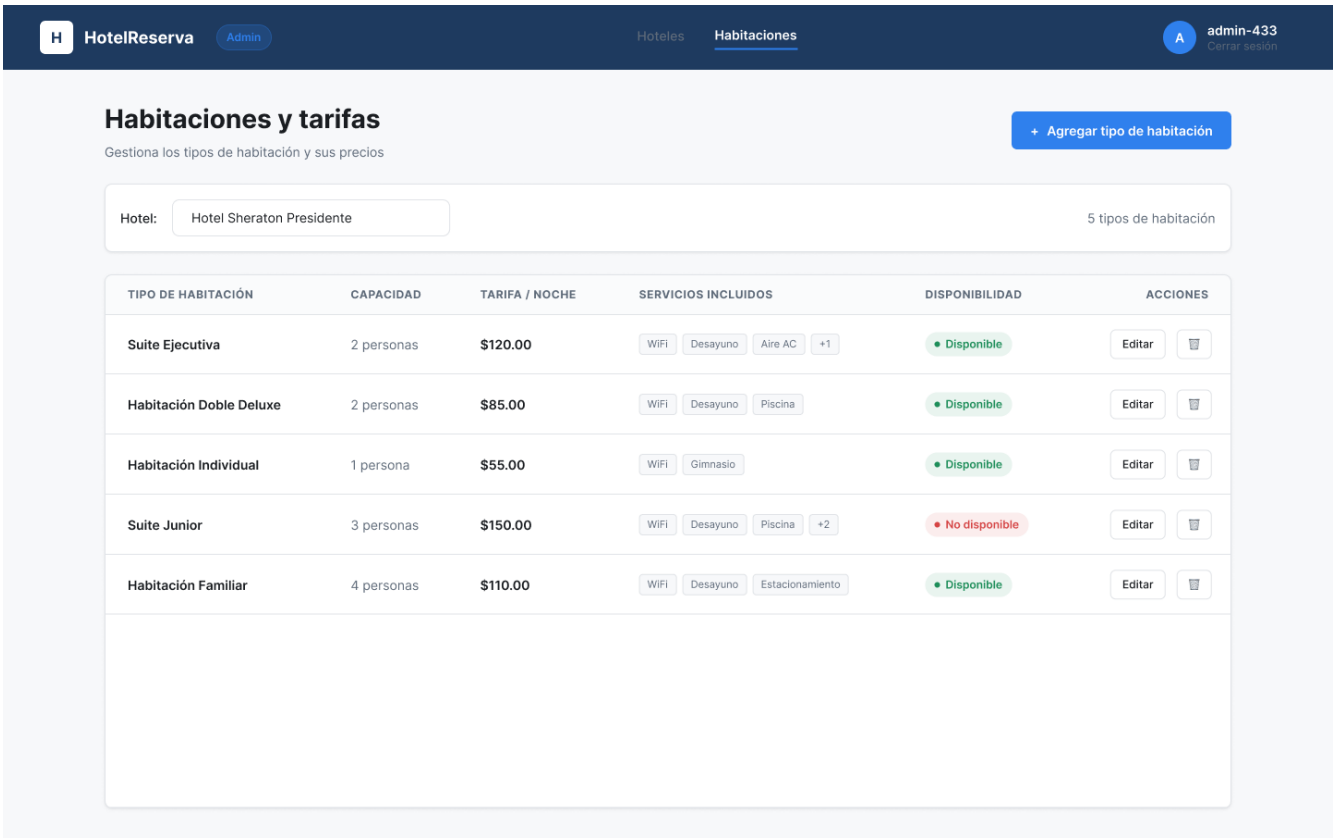


Figura 4. ADMIN - Habitaciones y tarifas

e) Pantalla 5

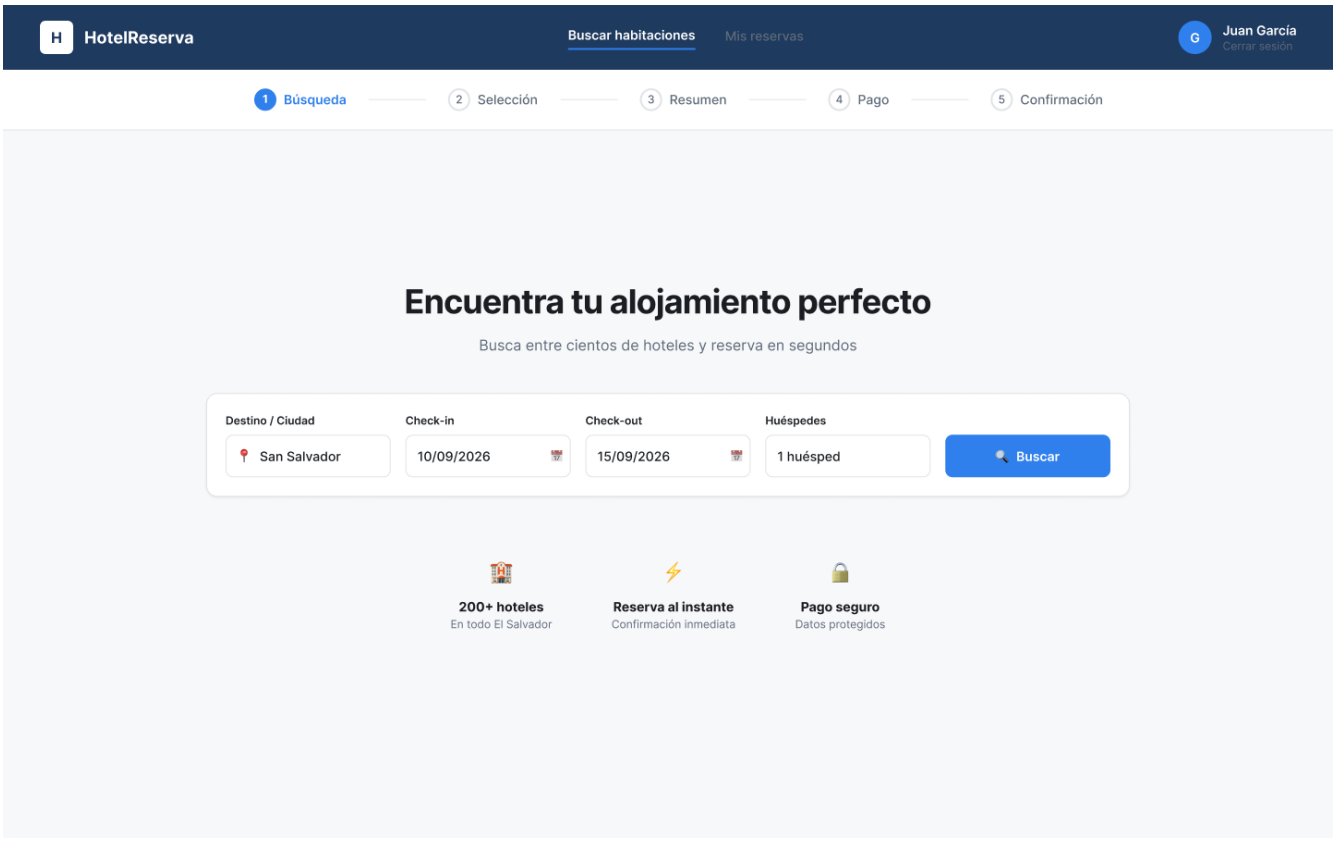


Figura 5. USER - Habitaciones y tarifas

f) Pantalla 6

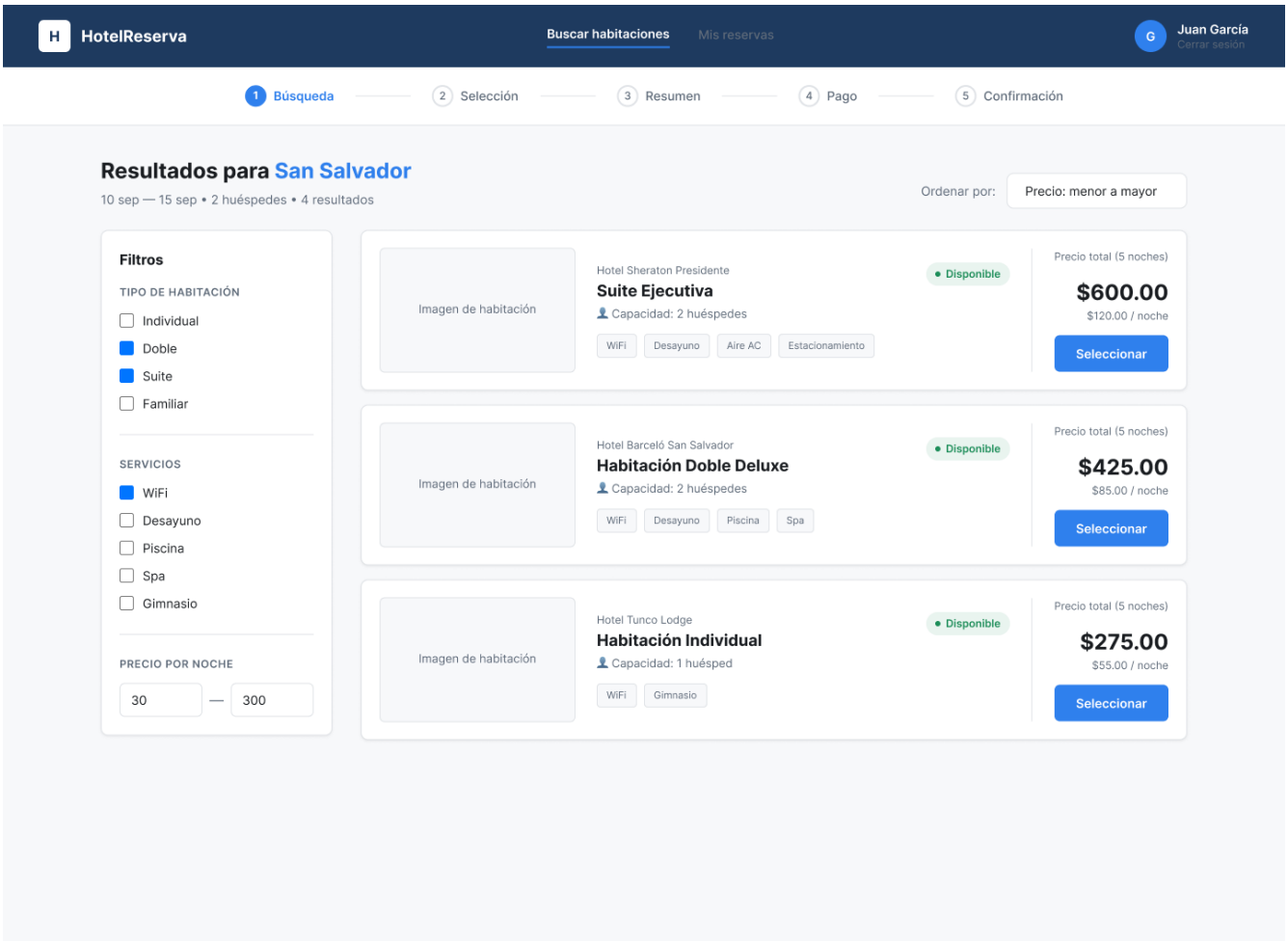


Figura 6. USER - Resultados de búsqueda

g) Pantalla 7

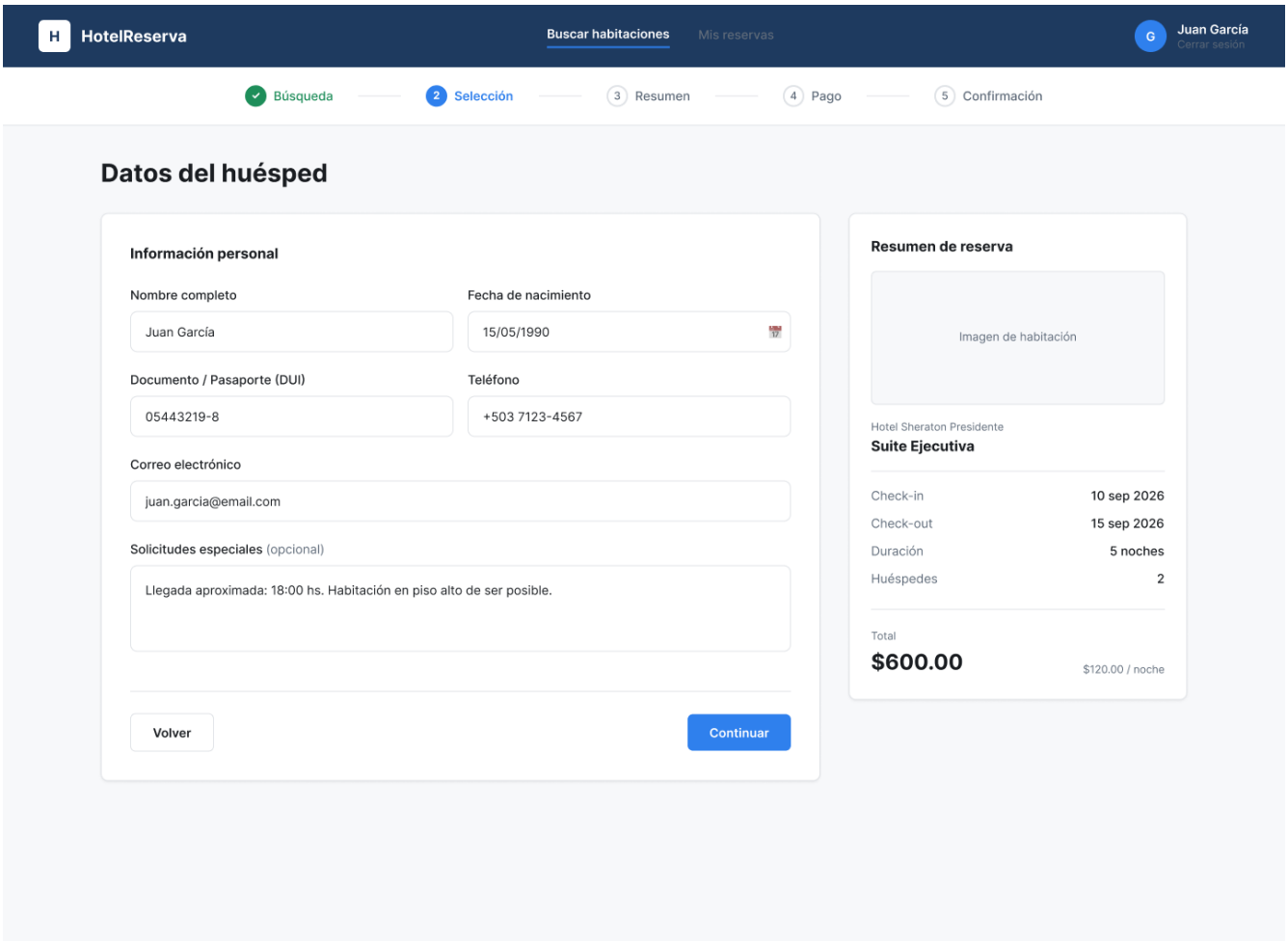


Figura 7. USER - Datos del huésped

h) Pantalla 8

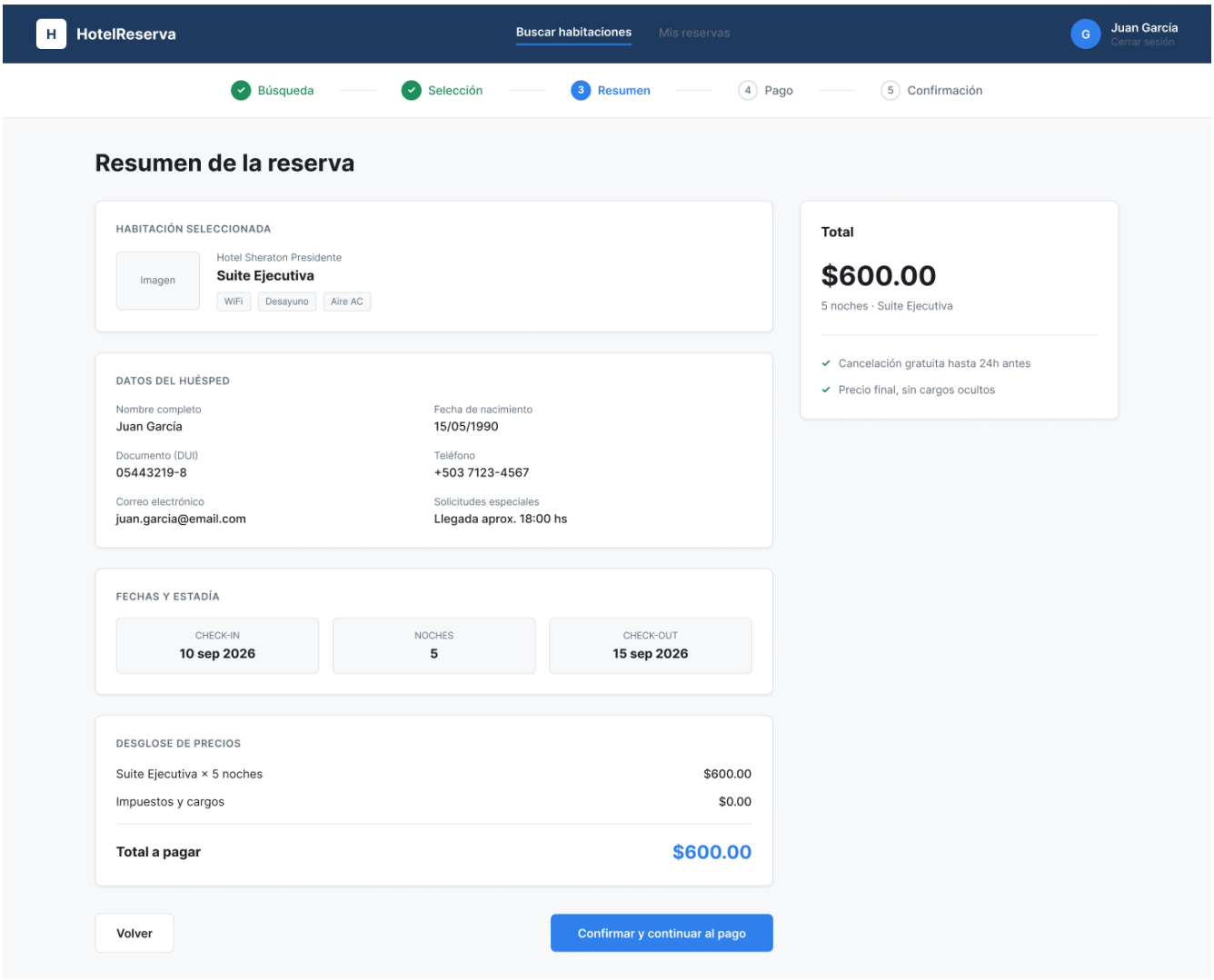


Figura 8. USER - Resumen de la reserva

i) Pantalla 9

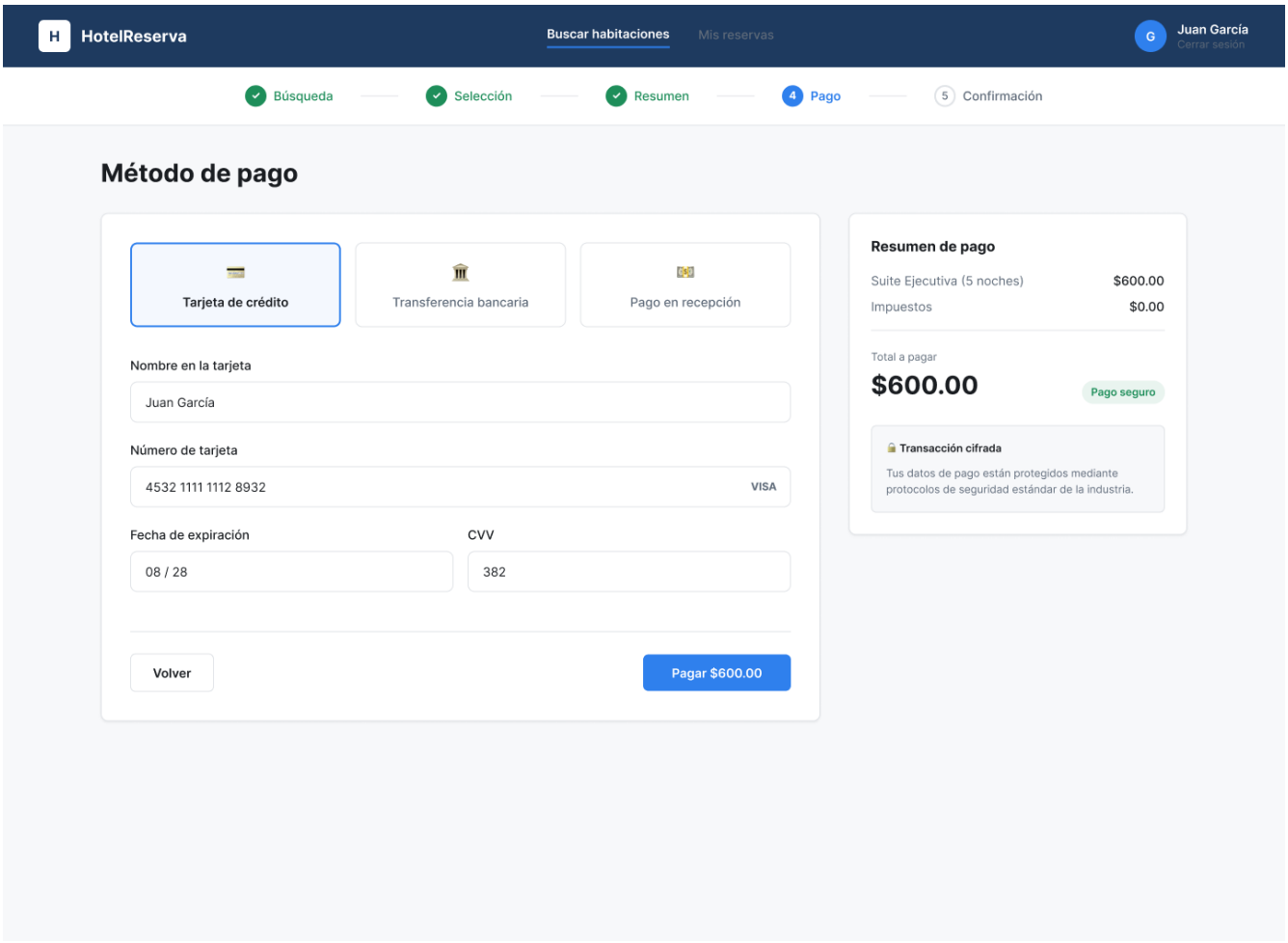


Figura 9. USER - Pago

j) Pantalla 10

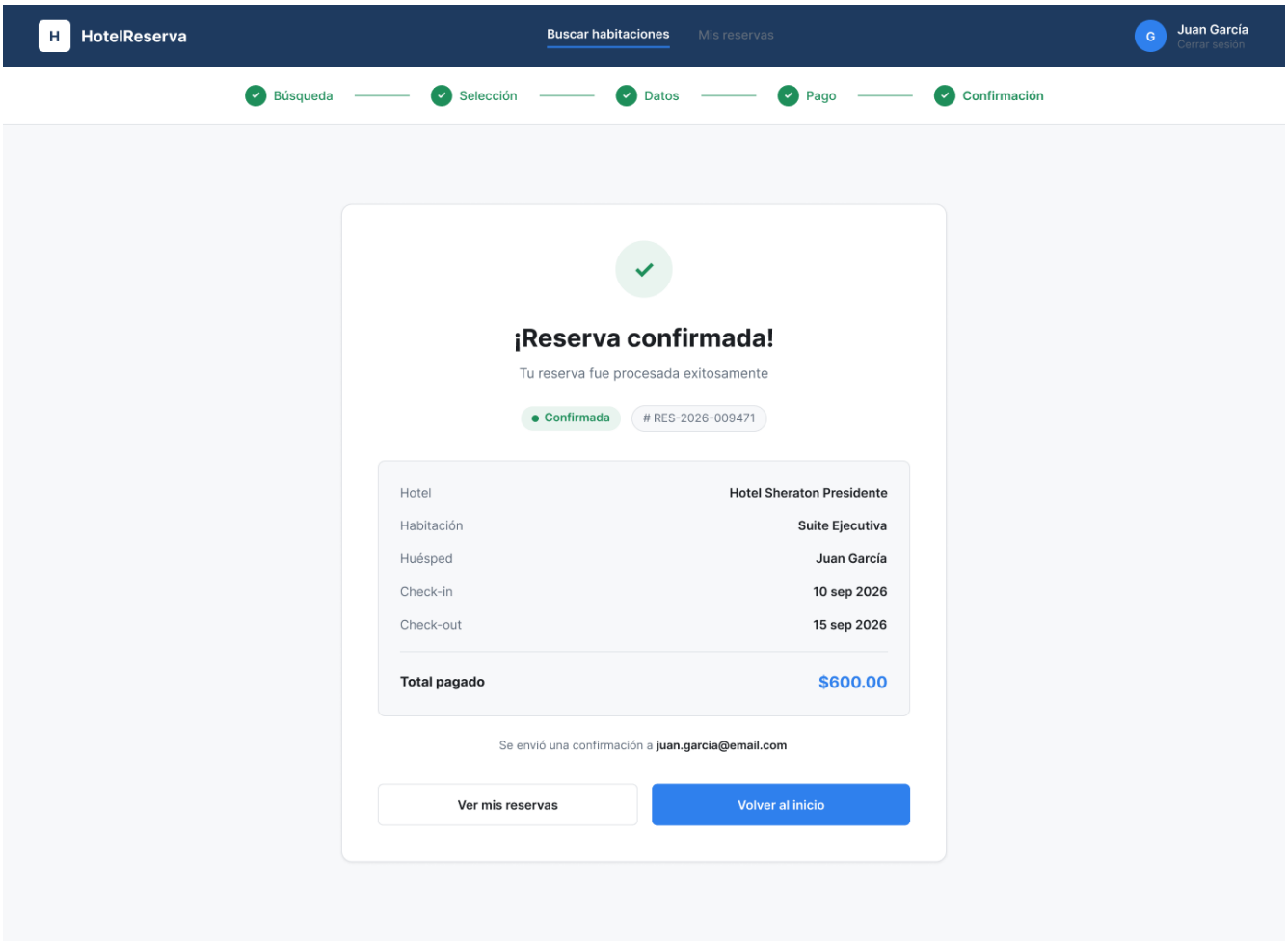


Figura 10. USER - Confirmación final

k) Pantalla 11

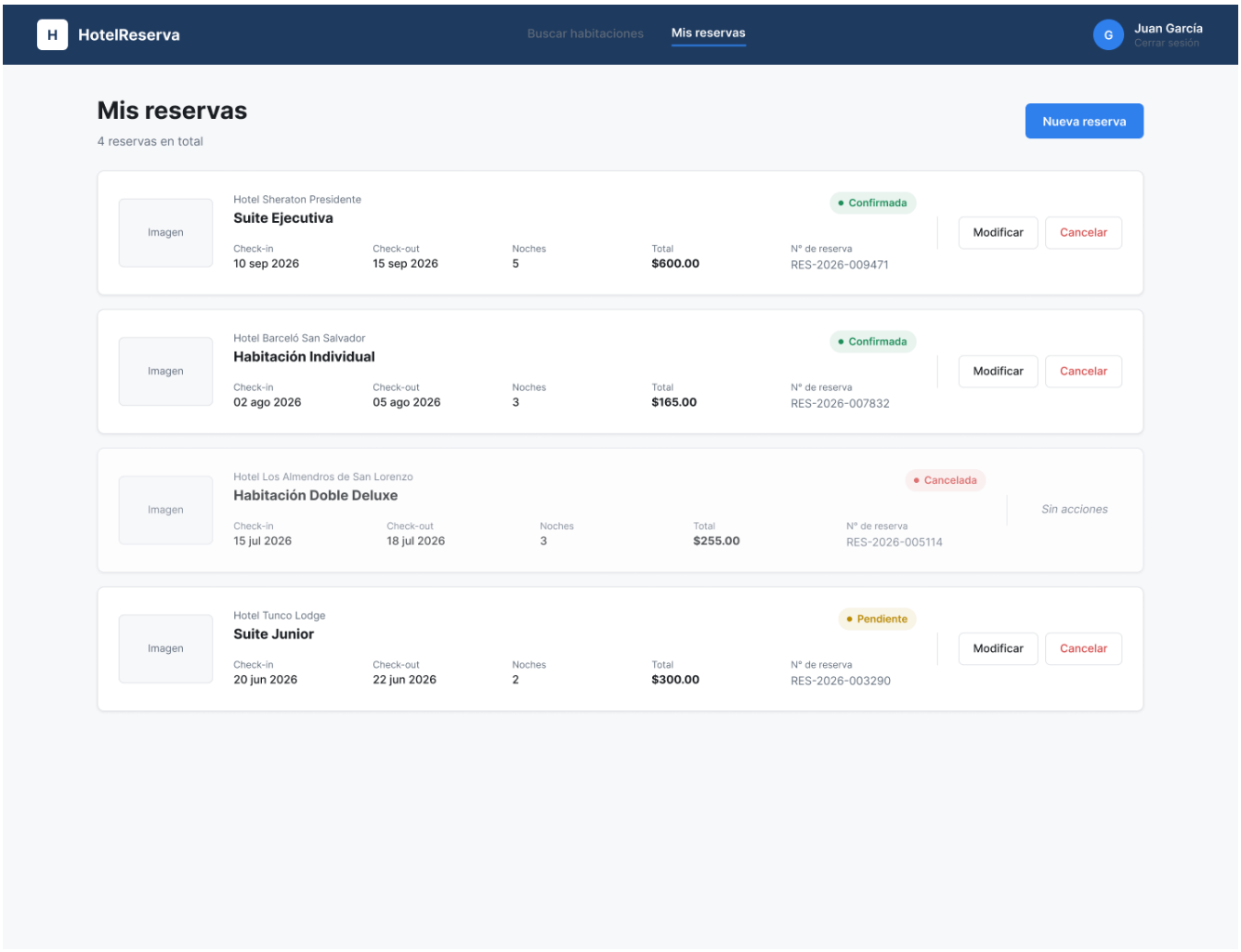


Figura 11. USER - Reservas

5.2 Gestión de la Base de Datos

a) Estructura de la Base de Datos

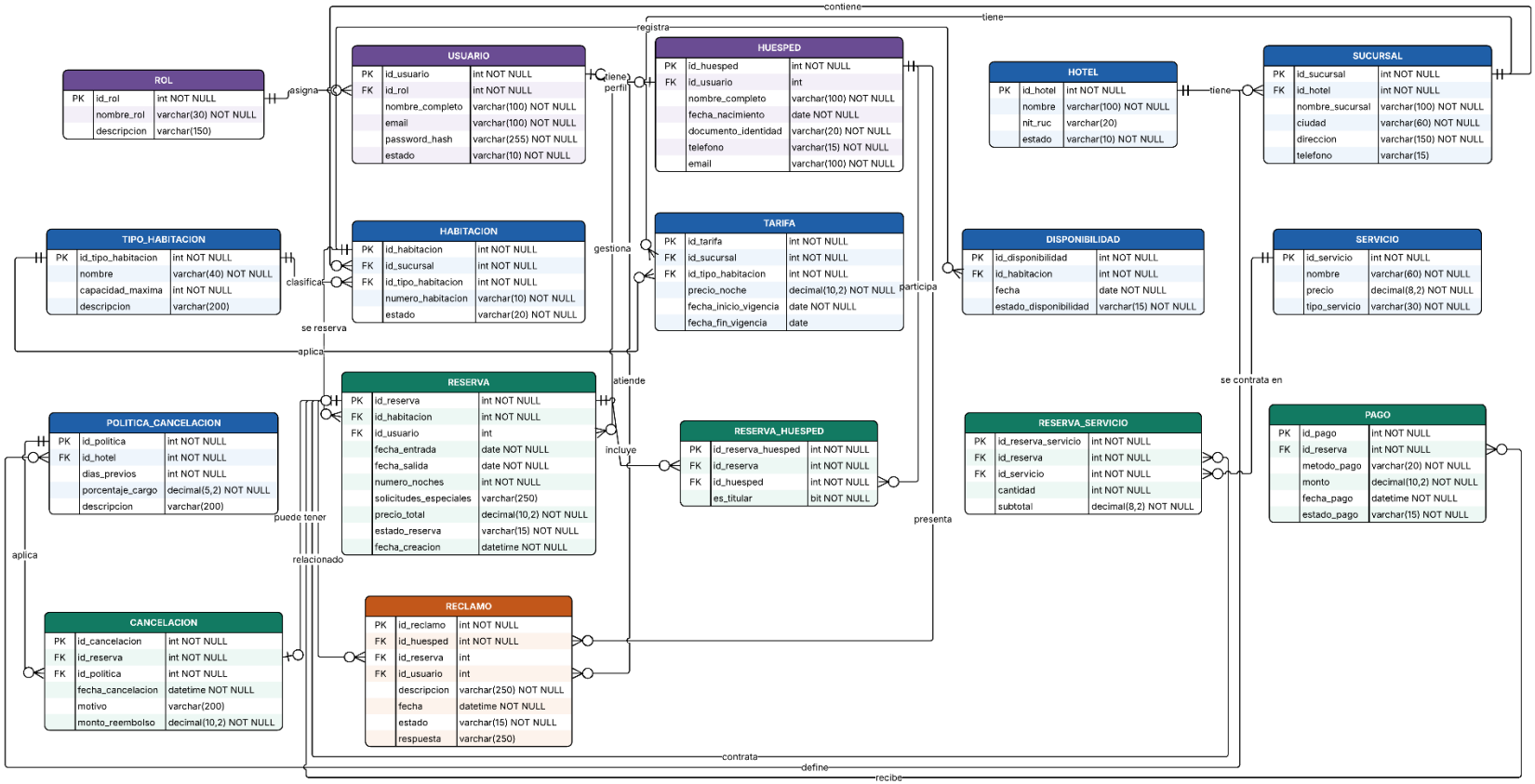


Figura 12. Diagrama entidad-relación de la base de datos

Nota: El enlace al diagrama en Lucidchart :

DiagramaER

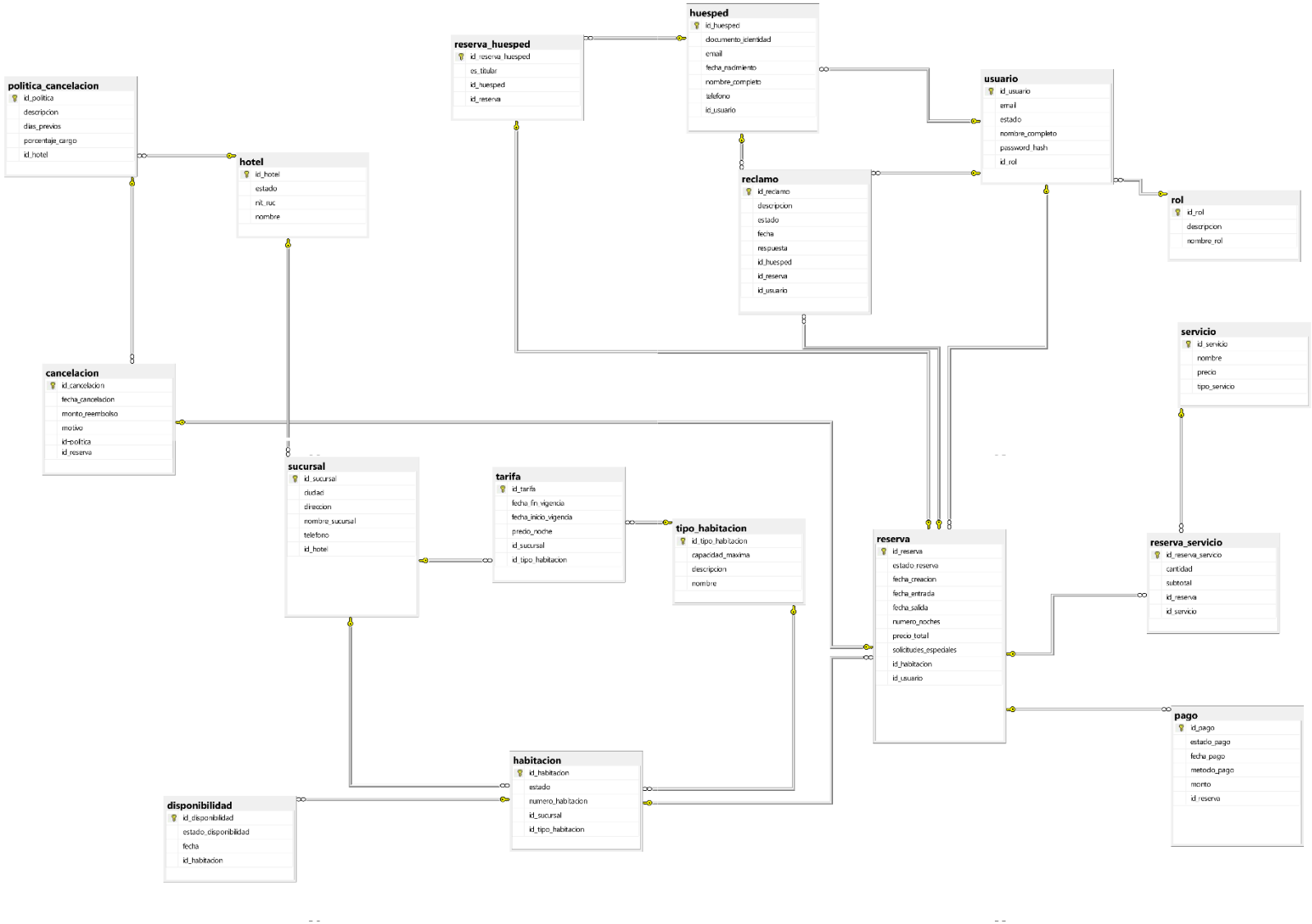


Figura 13. Modelo relacional de la base de datos en SQL Server

b) Diccionario de datos

1. Tabla ROL

Cataloga los roles de acceso al sistema.

Campo	Tipo de dato	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_rol	INT	Identificador del rol	-	Autonumérico	PK	Referenciado por usuario
nombre_rol	VARCHAR	Nombre del rol	30	ADMIN, RECEPCIONISTA, HUESPED	NOT NULL, UNIQUE	-
descripcion	VARCHAR	Detalle de funciones del rol	150	Texto libre	Permite nulos	-

2. Tabla USUARIO

Usuarios que inician sesión: empleados (administrador, recepcionista) y huéspedes con cuenta.

Campo	Tipo de dato	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_usuario	INT	Identificador del usuario	-	Autonumérico	PK	Referenciado por huesped, reserva, reclamo
id_rol	INT	Rol asignado	-	Existente en rol	NOT NULL, FK	FK → rol.id_rol
nombre_completo	VARCHAR	Nombre del usuario	100	Texto libre	NOT NULL	-
email	VARCHAR	Correo electrónico	100	Formato email	NOT NULL, UNIQUE	-
password_hash	VARCHAR	Contraseña cifrada	255	Hash (bcrypt)	NOT NULL	-
estado	VARCHAR	Estado de la cuenta	10	ACTIVO, INACTIVO	NOT NULL, default Activo	-

3. Tabla HOTEL
Cadena o propiedad hotelera principal.

Campo	Tipo de dato	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_hotel	INT	Identificador de la cadena/hotel	-	Autonumérico	PK	Referenciado por sucursal, politica_cancelacion
nombre	VARCHAR	Nombre comercial	100	Texto libre	NOT NULL	-
nit_ruc	VARCHAR	Identificación fiscal	20	Alfanumérico	UNIQUE, permite nulos	-
estado	VARCHAR	Estado del hotel	10	Activo, Inactivo	NOT NULL, default Activo	-

4. Tabla SUCURSAL
Sedes o ubicaciones físicas pertenecientes a un hotel.

Campo	Tipo de dato	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_sucursal	INT	Identificador de la sucursal	-	Autonumérico	PK	Referenciado por habitacion, tarifa
id_hotel	INT	Cadena a la que pertenece	-	Existente en hotel	NOT NULL, FK	FK → hotel.id_hotel
nombre_sucursal	VARCHAR	Nombre de la sede	100	Texto libre	NOT NULL	-
ciudad	VARCHAR	Ciudad/ubicación	60	Texto libre	NOT NULL	Usado como criterio de búsqueda
direccion	VARCHAR	Dirección física	150	Texto libre	NOT NULL	-
telefono	VARCHAR	Teléfono de contacto	15	Numérico	Permite nulos	-

5. Tabla TIPO_HABITACION
Catálogo de tipos de habitación disponibles.

Campo	Tipo de dato	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_tipo_habitacion	INT	Identificador del tipo	-	Autonumérico	PK	Referenciado por habitacion, tarifa
nombre	VARCHAR	Nombre del tipo	40	Individual, Doble, Suite, Familiar	NOT NULL	-
capacidad_maxima	INT	Huéspedes máximos	-	1-10	NOT NULL	-
descripcion	VARCHAR	Detalle del tipo	200	Texto libre	Permite nulos	-

6. Tabla HABITACION

Habitaciones físicas de cada sucursal.

Campo	Tipo de dato	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_habitacion	INT	Identificador de la habitación	-	Autonumérico	PK	Referenciado por disponibilidad, reserva
id_sucursal	INT	Sucursal a la que pertenece	-	Existente en sucursal	NOT NULL, FK	FK → sucursal.id_sucursal
id_tipo_habitacion	INT	Tipo de habitación	-	Existente en tipo_habitacion	NOT NULL, FK	FK → tipo_habitacion.id_tipo_habitacion
numero_habitacion	VARCHAR	Número/código físico	10	Alfanumérico	NOT NULL, UNIQUE por sucursal	-
estado	VARCHAR	Estado operativo	20	Disponible, Mantenimiento, Fuera de servicio	NOT NULL, default Disponible	-

7. Tabla TARIFA

Precios vigentes e históricos por tipo de habitación en cada sucursal.

Campo	Tipo de dato	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_tarifa	INT	Identificador de la tarifa	-	Autonumérico	PK	-
id_sucursal	INT	Sucursal que fija el precio	-	Existente en sucursal	NOT NULL, FK	FK → sucursal.id_sucursal
id_tipo_habitacion	INT	Tipo al que aplica	-	Existente en tipo_habitacion	NOT NULL, FK	FK → tipo_habitacion.id_tipo_habitacion
precio_noche	DECIMAL(10,2)	Precio por noche	-	Mayor a 0	NOT NULL	-
fecha_inicio_vigencia	DATE	Inicio de vigencia	-	Fecha válida	NOT NULL	-
fecha_fin_vigencia	DATE	Fin de vigencia	-	Fecha válida, mayor o igual al inicio	Permite nulos (tarifa vigente)	-

8. Tabla DISPONIBILIDAD

Estado de ocupación de cada habitación por día calendario.

Campo	Tipo de dato	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_disponibilidad	INT	Identificador del registro	-	Autonumérico	PK	-
id_habitacion	INT	Habitación referenciada	-	Existente en habitacion	NOT NULL, FK	FK → habitacion.id_habitacion
fecha	DATE	Día calendario	-	Fecha válida	NOT NULL, UNIQUE con id_habitacion	-
estado_disponibilidad	VARCHAR	Estado del día	15	Disponible, Ocupada, Bloqueada	NOT NULL	-

9. Tabla HUESPED

Clientes que realizan reservas.

Campo	Tipo de dato	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_huesped	INT	Identificador del huésped	-	Autonumérico	PK	Referenciado por reserva_huesped, reclamo
id_usuario	INT	Cuenta de acceso del huésped, si la tiene	-	Existente en usuario	Permite nulos, FK	FK → usuario.id_usuario
nombre_completo	VARCHAR	Nombre del huésped	100	Texto libre	NOT NULL	-
fecha_nacimiento	DATE	Fecha de nacimiento	-	Fecha válida, pasada	NOT NULL	-
documento_identidad	VARCHAR	DUI/Pasaporte	20	Alfanumérico	NOT NULL, UNIQUE	-
telefono	VARCHAR	Teléfono	15	Numérico	NOT NULL	-
email	VARCHAR	Correo electrónico	100	Formato email	NOT NULL	-

10. Tabla SERVICIO

Catálogo de servicios adicionales ofrecidos (alimentación, transporte, lavandería, etc.).

Campo	Tipo de dato	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_servicio	INT	Identificador del servicio	-	Autonumérico	PK	Referenciado por reserva_servicio
nombre	VARCHAR	Nombre del servicio	60	Texto libre	NOT NULL	-
precio	DECIMAL(8,2)	Costo del servicio	-	Mayor o igual a 0	NOT NULL	-
tipo_servicio	VARCHAR	Categoría	30	Alimentación, Transporte, Lavandería, Spa, Otro	NOT NULL	-

11. Tabla RESERVA

Registro central del proceso de reserva de una habitación.

Campo	Tipo de dato	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_reserva	INT	Identificador de la reserva	-	Autonumérico	PK	Referenciado por reserva_huesped, pago, cancelacion, reserva_servicio, reclamo
id_habitacion	INT	Habitación reservada	-	Existente en habitacion	NOT NULL, FK	FK → habitacion.id_habitacion. El hotel se obtiene por habitación → sucursal → hotel
id_usuario	INT	Usuario que gestiona	-	Existente en usuario	Permite nulos, FK	FK → usuario.id_usuario
fecha_entrada	DATE	Check-in	-	Fecha válida	NOT NULL	-
fecha_salida	DATE	Check-out	-	Fecha válida, mayor a entrada	NOT NULL	-
numero_noches	INT	Noches calculadas	-	Mayor o igual a 1	NOT NULL	Calculado (salida - entrada)
solicitudes_especiales	VARCHAR	Peticiones del huésped	250	Texto libre	Permite nulos	-
precio_total	DECIMAL(10,2)	Total de la reserva	-	Mayor o igual a 0	NOT NULL	Suma tarifa + servicios

estado_reserva	VARCHAR	Estado actual	15	Pendiente, Confirmada, Cancelada, Finalizada	NOT NULL, default Pendiente	-
fecha_creacion	TIMESTAMP	Registro de creación	-	Fecha/hora válida	NOT NULL, default now()	-

12. Tabla RESERVA_HUESPED

Campo	Tipo de dato	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_reserva_huesped	INT	Identificador del registro	-	Autonumérico	PK	Tabla puente N:M
id_reserva	INT	Reserva asociada	-	Existente en reserva	NOT NULL, FK	FK → reserva.id_reserva
id_huesped	INT	Huésped que se hospeda	-	Existente en huesped	NOT NULL, FK	FK → huesped.id_huesped
es_titular	BIT	Indica si es el titular	-	0 o 1	NOT NULL, default 0; solo una fila con valor 1 por reserva	-

13. Tabla RESERVA_SERVICIO

Tabla puente N:M entre reserva y servicio.

Campo	Tipo de dato	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_reserva_servicio	INT	Identificador del registro	-	Autonumérico	PK	Tabla puente N:M
id_reserva	INT	Reserva asociada	-	Existente en reserva	NOT NULL, FK	FK → reserva.id_reserva
id_servicio	INT	Servicio contratado	-	Existente en servicio	NOT NULL, FK	FK → servicio.id_servicio
cantidad	INT	Cantidad solicitada	-	Mayor o igual a 1	NOT NULL, default 1	-
subtotal	DECIMAL(8,2)	Costo total del servicio	-	Mayor o igual a 0	NOT NULL	cantidad x precio unitario

14. Tabla PAGO

Pagos asociados a una reserva.

Campo	Tipo de dato	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_pago	INT	Identificador del pago	-	Autonumérico	PK	-
id_reserva	INT	Reserva pagada	-	Existente en reserva	NOT NULL, FK	FK → reserva.id_reserva
metodo_pago	VARCHAR	Forma de pago	20	Tarjeta, Efectivo, Transferencia	NOT NULL	-
monto	DECIMAL(10,2)	Monto pagado	-	Mayor a 0	NOT NULL	-
fecha_pago	TIMESTAMP	Fecha/hora del pago	-	Fecha/hora válida	NOT NULL, default now()	-
estado_pago	VARCHAR	Estado de la transacción	15	Pendiente, Completado, Rechazado	NOT NULL	-

15. Tabla POLITICA_CANCELACION

Reglas de cancelación definidas por cada hotel.

Campo	Tipo de dato	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_politica	INT	Identificador de la política	-	Autonumérico	PK	-
id_hotel	INT	Hotel al que aplica	-	Existente en hotel	NOT NULL, FK	FK → hotel.id_hotel
dias_previos	INT	Días de anticipación exigidos	-	Mayor o igual a 0	NOT NULL	-
porcentaje_cargo	DECIMAL(5,2)	% de cargo por cancelar tarde	-	0 a 100	NOT NULL	-
descripcion	VARCHAR	Detalle de la política	200	Texto libre	Permite nulos	-

16. Tabla CANCELACION

Registro de cancelaciones de reservas y su respectivo cálculo de reembolso.

Campo	Tipo de dato	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_cancelacion	INT	Identificador	-	Autonumérico	PK	-
id_reserva	INT	Reserva cancelada	-	Existente en reserva	NOT NULL, FK, UNIQUE	FK → reserva.id_reserva
id_politica	INT	Política aplicada	-	Existente en politica_cancelacion	NOT NULL, FK	FK → politica_cancelacion.id_politica
fecha_cancelacion	TIMESTAMP	Fecha del evento	-	Fecha/hora válida	NOT NULL, default now()	-
motivo	VARCHAR	Razón de cancelación	200	Texto libre	Permite nulos	-
monto_reembolso	DECIMAL(10,2)	Monto a reembolsar	-	Mayor o igual a 0	NOT NULL	-

17. Tabla RECLAMO

Solicitudes y reclamos presentados por los huéspedes.

Campo	Tipo de dato	Descripción	Tamaño	Valores permitidos / rango	Restricciones	Relaciones
id_reclamo	INT	Identificador del reclamo	-	Autonumérico	PK	-
id_huesped	INT	Huésped que reclama	-	Existente en huesped	NOT NULL, FK	FK → huesped.id_huesped
id_reserva	INT	Reserva relacionada	-	Existente en reserva	Permite nulos, FK	FK → reserva.id_reserva
id_usuario	INT	Empleado que atiende	-	Existente en usuario	Permite nulos, FK	FK → usuario.id_usuario
descripcion	VARCHAR	Detalle del reclamo	250	Texto libre	NOT NULL	-
fecha	TIMESTAMP	Fecha del reclamo	-	Fecha/hora válida	NOT NULL, default now()	-
estado	VARCHAR	Estado de atención	15	Abierto, En proceso, Cerrado	NOT NULL, default Abierto	-
respuesta	VARCHAR	Resolución dada	250	Texto libre	Permite nulos	-

6. Resultados y Conclusiones

6.1 Resultados Obtenidos

Como resultado de esta primera fase, el equipo definió el diseño del sistema de administración y reserva de habitaciones hoteleras y dejó lista la base para la Fase 2. Se logró:

- Redactar el objetivo general y los nueve objetivos específicos del sistema.
- Elegir un stack de herramientas gratuitas (Java con Spring Boot, SQL Server, HTML/CSS/JavaScript, Figma, GitHub y Trello) que sea viable en los tiempos del curso.
- Diseñar el modelo relacional con las tablas necesarias para reservas, pagos, cancelaciones, reclamos, usuarios y roles.
- Elaborar 11 pantallas de prototipo en Figma como guía para el frontend.
- Organizar al equipo con GitHub (control de versiones) y Trello (seguimiento de tareas), con roles y canales de comunicación definidos.

6.2 Conclusiones

En esta fase el equipo comprobó que definir el alcance, las tablas y los prototipos antes de programar reduce los cambios durante la implementación.

Los principales retos fueron coordinar los horarios de tres personas en un plazo corto y elegir un control de acceso simple por roles, en lugar de un mecanismo de autenticación más robusto, para ajustarse al tiempo disponible.

El principal aprendizaje fue aplicar una metodología ágil simplificada con un tablero de Trello, para ver el avance de cada tarea en todo momento. En la Fase 2 se implementó Spring Security con JWT y la interfaz se conectó con la API REST.

7. Anexos y Documentación de Apoyo

En esta sección se incluye documentación complementaria que respalda el desarrollo del proyecto:

Anexo 1: Tablero de Trello.

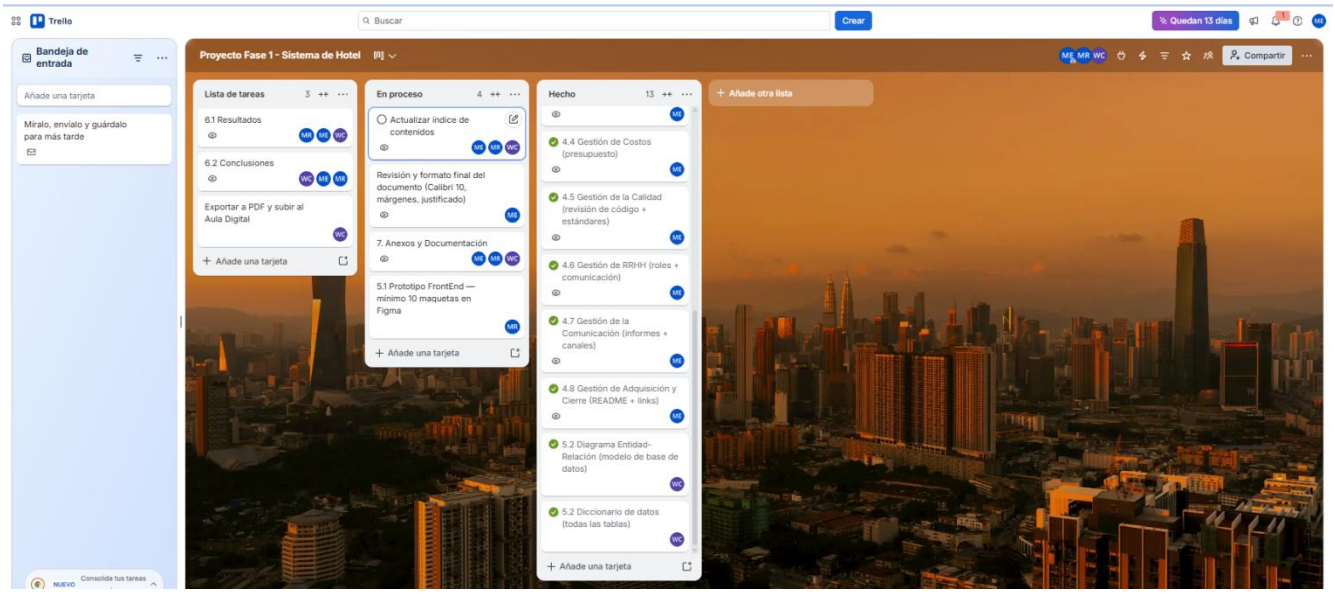


Figura 14. Tablero de Trello con las tareas del equipo en proceso.

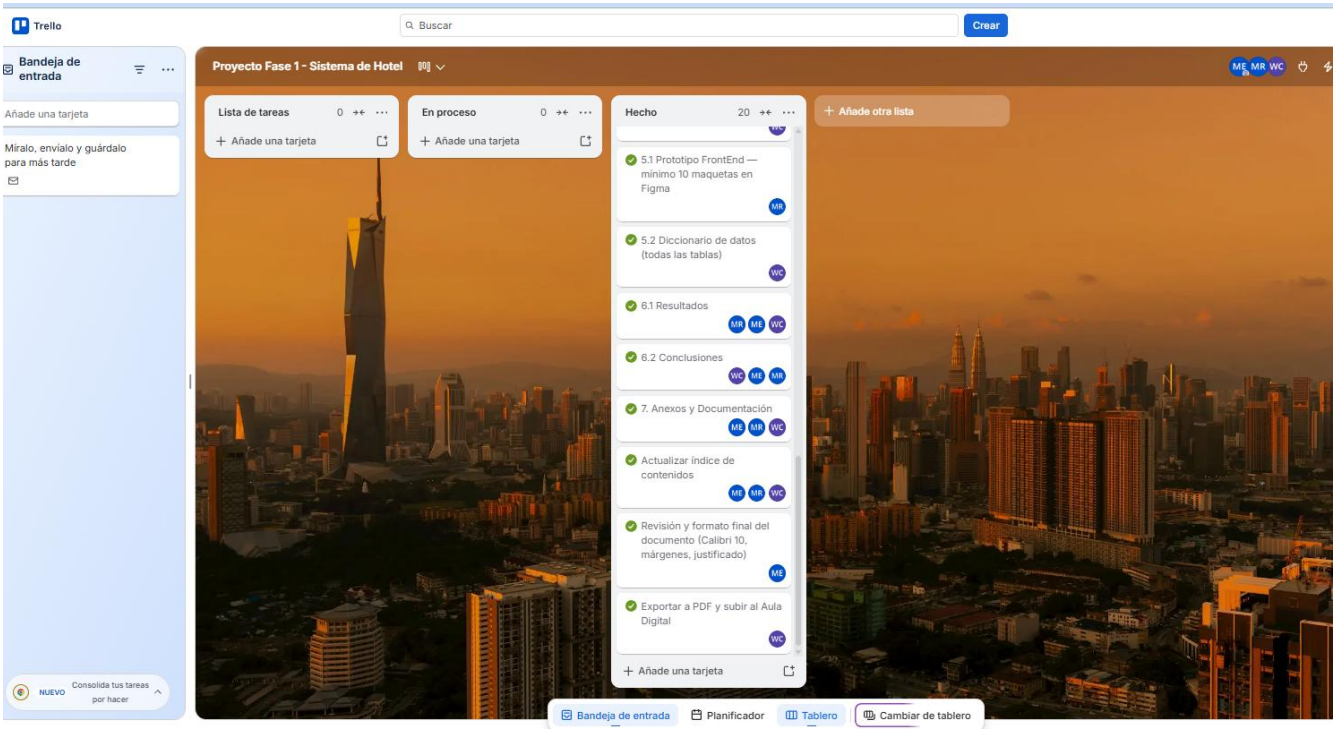


Figura 15. Tablero de Trello con todas las tareas ya completadas.

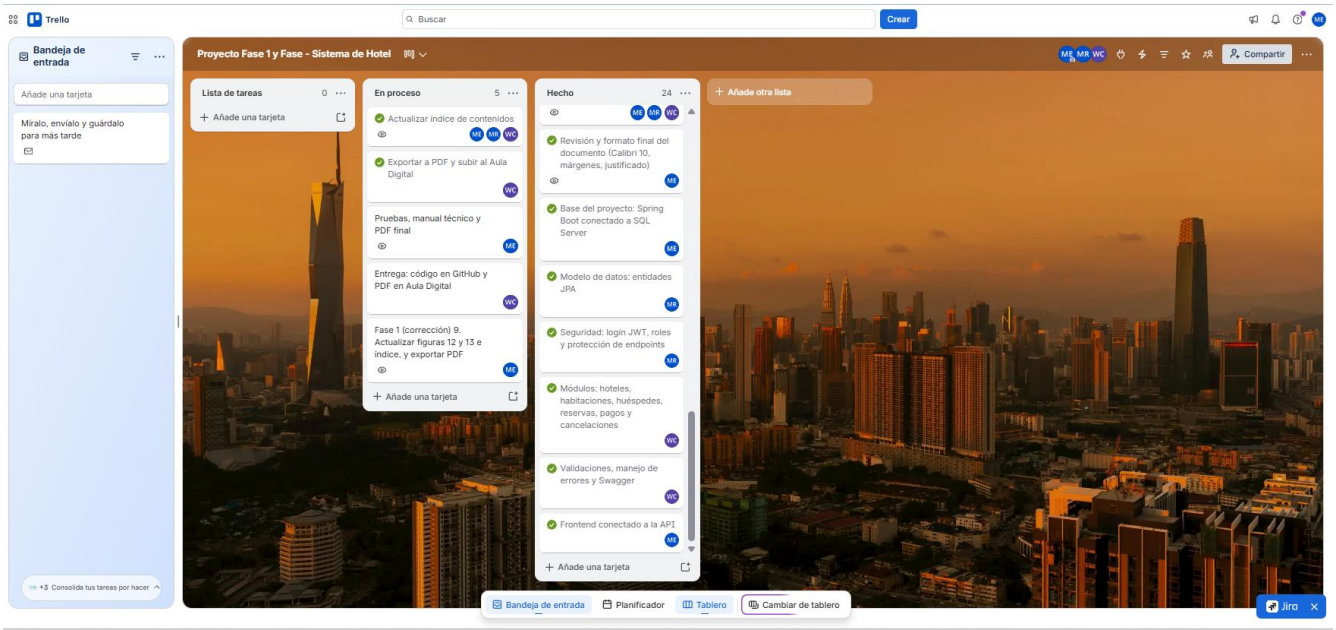


Figura 16. Tablero de Trello con las tareas de la fase final del proyecto en proceso.

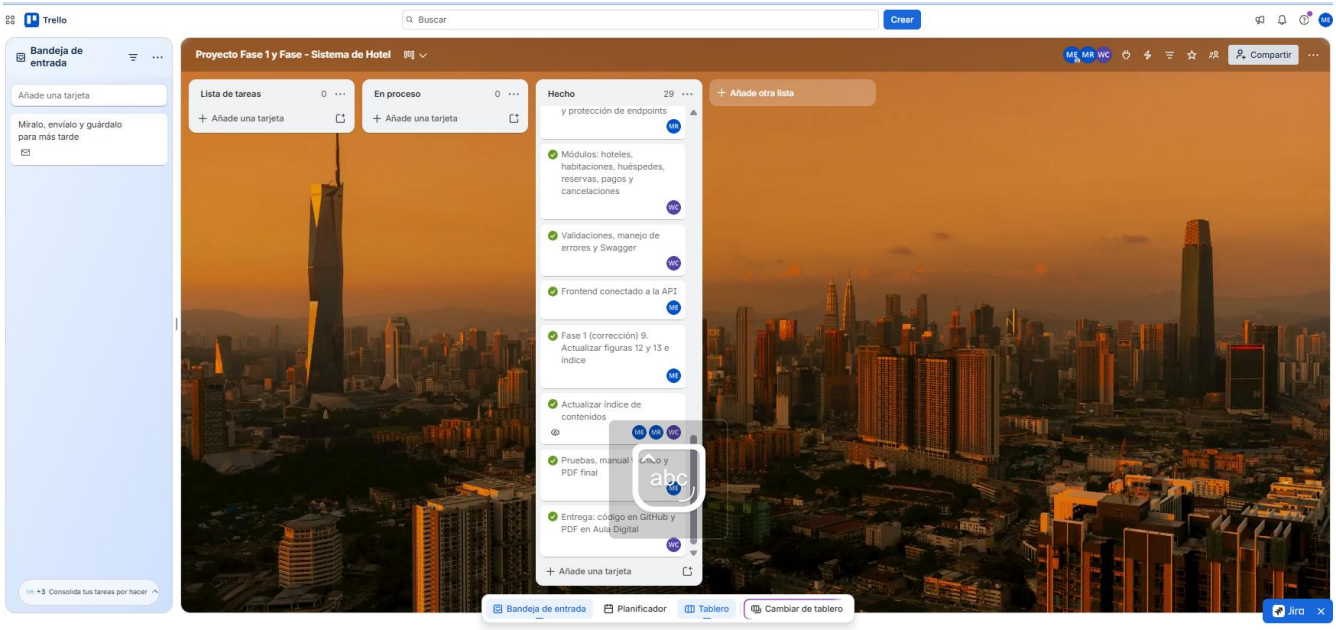


Figura 17. Tablero de Trello con todas las tareas de la fase final ya completadas.

Anexo 2: Repositorio de GitHub.

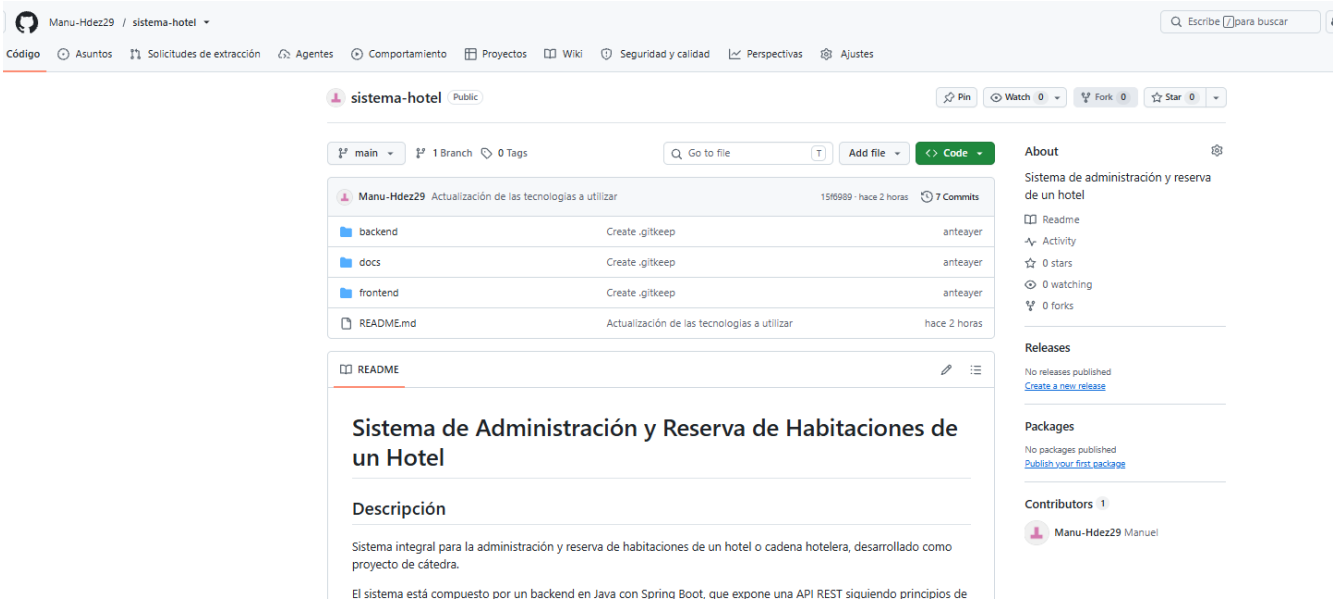


Figura 18. Estructura de carpetas del repositorio en GitHub.

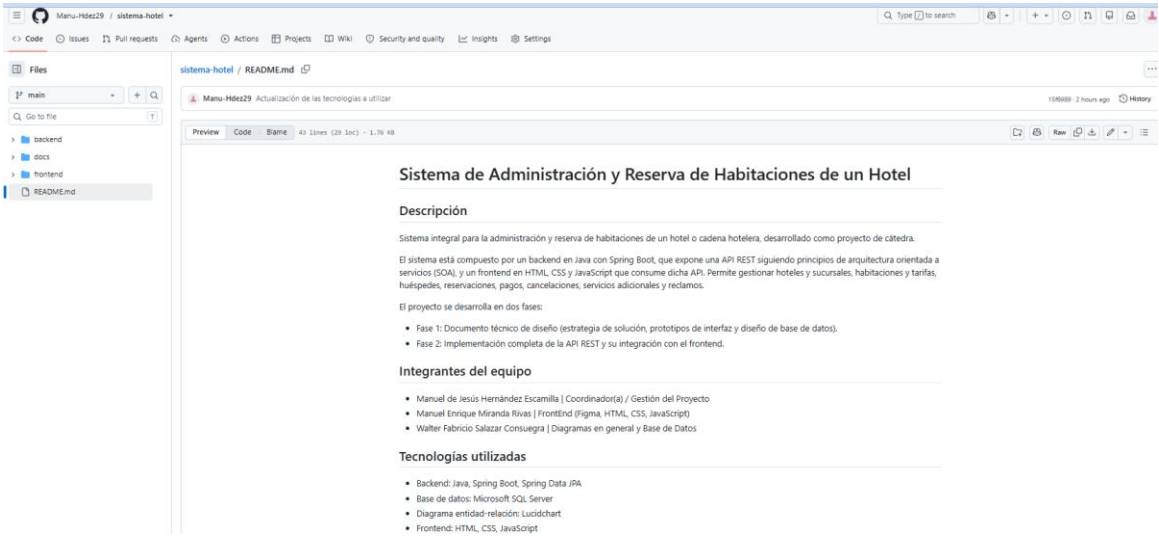


Figura 19. Archivo README.md del proyecto.

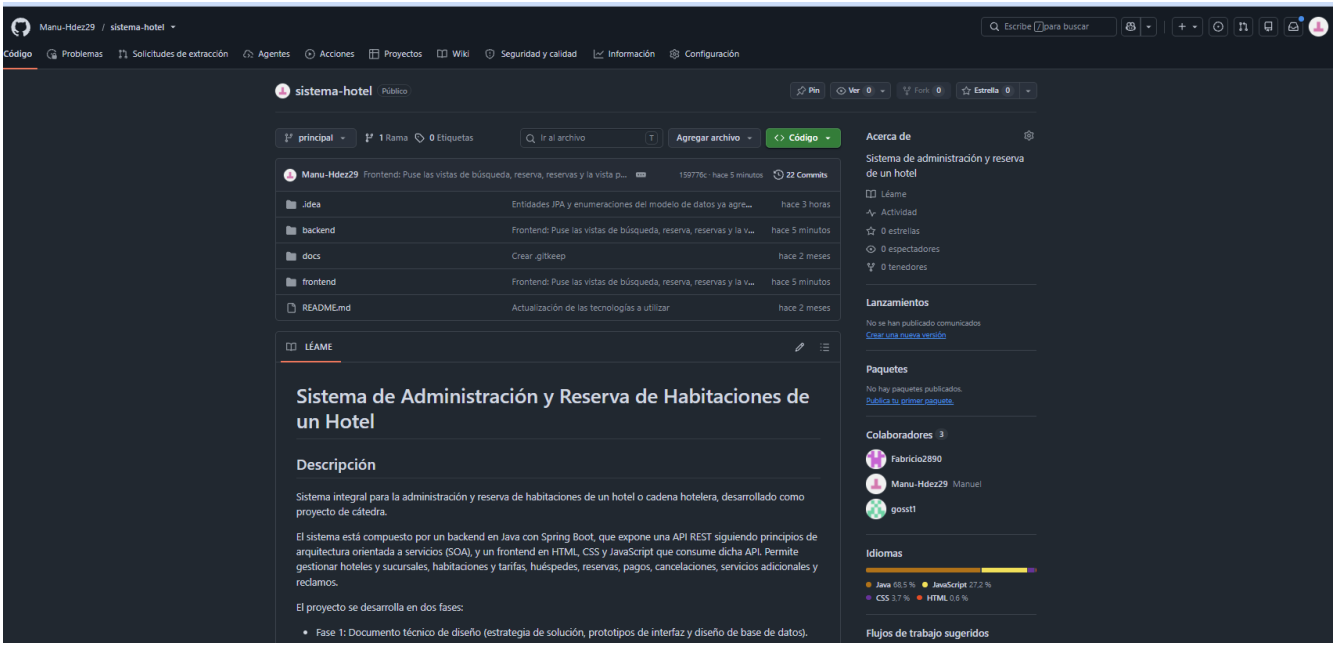


Figura 20. Estructura de carpetas del repositorio en GitHub con el proyecto ya finalizado.